Chemical Reaction Prediction

Ravi Shende
University of California, Irvine
Irvine, California, USA
rshende@uci.edu

Jae Choi
University of California, Irvine
Irvine, California, USA
jaehyuc3@uci.edu

Paul Schroller
University of California, Irvine
Irvine, California, USA
pschroll@uci.edu

Andrew Zheng
University of California, Irvine
Irvine, California, USA
zhenga10@uci.edu

Qihua Xue
University of California, Irvine
Irvine, California, USA
qihuax1@uci.edu

1 Abstract

The modern development of pharmaceuticals, materials, and environmental models requires understanding how chemicals interact to produce the products and byproducts that are being studied. Chemical reaction prediction is crucial in these areas, because manually running reaction experiments on vast combinations of reactants and reagents is infeasible with limited resources and time [10]. We examine a variety of approaches to predict chemical reaction, evaluating graph-based and seq-to-seq based approaches. Among the considered models, a seq-to-seq transformer achieves the highest prediction accuracy. We further assess the model's ability to generalize using a secondary dataset, and we test if augmenting the data with random SMILES improves prediction accuracy.

2 Introduction

In this paper, we apply deep learning to predict chemical reactions. Given one or more reactants and reagents in SMILES [12] format as input, our sequence to sequence models output a predicted product or products. SMILES is a method of representing structures of chemical molecules in a compact string format. It is commonly used in machine learning applications for those reasons.

The key metrics involved with this experiment are exact match accuracy, token accuracy, and valid SMILES accuracy. An exact match means that the model was able to predict a product chemically equivalent to the ground truth. Because there can be multiple valid SMILES representations of the same molecule, this does not mean that the predicted SMILES string equals the ground truth SMILES string. Instead, we canonicalize both strings and compare those versions. This ensures that correct predictions are not penalized for being in a different order. Token accuracy measures the percentage of tokens in the correct location. Because tokens can change based on the order of the SMILES strings, token accuracy does directly compare the two SMILES strings. As a result, it is possible to have an exact-match accuracy of 100% with a token accuracy of less than 100%. Finally, the valid SMILES prediction rate indicates whether the SMILES string output by the model can be mapped to a valid molecule. Whether this molecule is equivalent to the target molecule is not considered by this metric. Instead, it acts as a measure of whether the model learns how to produce syntactically and chemically valid SMILES representations. With these three metrics, we can compare how the models perform at predicting the results of chemical reactions.

3 Data

The two datasets used were the USPTO-MIT [6] and ORDerly benchmark [13] datasets. All models are trained on the USPTO-MIT dataset. The ORDerly dataset is used to test how well the transformer model generalizes to different datasets. Both datasets are cleaned versions of larger datasets (USPTO and ORD) to have targets for all source strings. This allows us to run supervised learning models on the data without dealing with self-supervised or unsupervised learning of potentially messy data. While this does decrease the overall size of the data, the smaller datasets still have a substantial number of samples, and their more limited size is ideal for reducing compute costs. The USPTO-MIT dataset has 409035 training reactions, 30000 validation reactions, and 40000 test reactions. The ORDerly dataset has 86232 test reactions.

4 Architecture Design and Training

4.1 Baseline

The baseline network was a sequence to sequence RNN model. It did not use GRU, LSTM, or attention. The hyperparameters were tuned with Optuna and {'hid_dim': 256, 'n_layers': 2, 'dropout': 0.3, 'lr': 0.0001, 'tf_ratio': 0.7} performed the best. The model was trained over 10 epochs, and the tf_ratio was reduced each epoch to teach the model how to handle wrong predictions.

4.2 Recursive Neural Network

To improve over the baseline sequence-to-sequence recurrent model, we implemented a recursive neural network (Recursive NN) architecture for chemical reaction prediction. The model receives reaction SMILES as input and generates the corresponding product SMILES autoregressively.

The preprocessing pipeline first removes atom-mapping annotations from the USPTO-MIT dataset and tokenizes reaction strings into character-level representations. The encoder recursively composes token embeddings into higher-level latent representations using a tree-structured composition process. The decoder is implemented as a gated recurrent unit (GRU), which sequentially predicts output tokens until generating the final product SMILES.

Compared with the baseline encoder–decoder RNN, the Recursive NN was designed to better capture hierarchical structure and long-range dependencies within reaction sequences. Training and evaluation were performed using token-level prediction and exact-match evaluation.

To optimize training performance, we performed hyperparameter tuning using Ray Tune [9]. Candidate configurations varied batch size, learning rate, dropout, embedding dimension, and hidden dimension. Selection was based primarily on validation loss and validation token accuracy.

The selected configuration was:

- Batch size: 32
- Learning rate: 8.58×10^{-4}
- Dropout: 0.088
- Embedding dimension: 128
- Hidden dimension: 512

The model was trained for 5 epochs.

4.3 Graph Neural Network

Our first graph-based approach was a simple Graph Neural Network (GNN) with multi-head attention. The graph representations of the reactants and reagents are passed to the GNN, which uses two convolution layers to process the graphs, then two multilayer perceptron (MLP) layers to predict the product graph. To generate this product graph prediction, the GNN produces a prediction for each pair of nodes in the input graph, predicting whether they will have an edge connecting them in the output, and what type of bond this edge will represent.

The natural choice for the loss function is cross-entropy loss, because the model makes multiclass predictions. However, the sparsity of molecular graphs raises a significant class-imbalance issue. The overwhelming majority of pairs of atoms in a molecule will not have a bond between them, so the model might learn to simply predict that there will be no bond for all atom pairs, because this prediction will be correct the overwhelming majority of the time, allowing the model to reduce the loss without learning anything meaningful. To address this issue, we added weights to the cross entropy loss, which means that false negatives incur a more severe loss than false positives for the less common classes by a factor calculated using the ratio of atom pairs with bonds to total atom pairs. With the loss function balanced correctly, the model cannot reduce loss by simply making blanket predictions, and instead must truly learn the graph structure to make meaningful chemical reaction predictions.

After tuning hyperparameters with Ray Tune, the model trained for 10 epochs, with a hidden layer size of 128, a learning rate of 1.9×10^{-4}, and a batch size of 32.

4.4 Graph Neural Network + Transformer

We used a graph-to-sequence architecture for the prediction of chemical reaction products. Instead of treating the reactants only as text, we converted the input reactant SMILES into a molecular graph using RDKit [7] and PyTorch Geometric [3]. In this graph representation, atoms are treated as nodes, and chemical bonds are treated as edges like in Table 1,

Table 1: Graph Representation

Features	representations
Atom Features	Atomic number, degree, formal charge, aromaticity
Bond Features	Single bond, double bond, triple bond, aromatic bond, conjugation, ring membership

For this model, we use the graph neural network model based encoder and the transformer model based decoder. The GNN encoder will learn the atom level representation, which contains the information about atoms and bounds. The input graph will be transformed into a hidden vector, and it will be used as cross attention for the transformer decoder.

The Transformer decoder will predict the reaction and generate the product Smiles one token at a time. In the decoder, masked self-attention allows the product tokens to attend to previous product tokens and learn SMILES syntax patterns such as branches, ring closures and atom sequences. Cross-attention from encoder then allows the decoder conditioned to generating product on the reactant molecular graph. The decoder will generate the predictions over the SMILES vocabulary. During this process, decoder will choose the highest probability token at each step then it will generate the reaction product.

The model was supervised trained to predict the product SMILES sequence from the reactants molecular graph. The dataset has the reactant and product in SMILES syntax. During training, it will back propagate after checking the performance each time. Model performance is monitored using loss, token accuracy, valid SMILES Rate, and canonical exact match.

Table 2: Model training metrics

Metric	Description
Loss	Measures how different the prediction from the actual target data during training
Token Accuracy	Measures how many individual SMILES are predicted correctly. It compares each token in predict and tartget SMILES
Valid SMILES Rate	Measures how many generated SMILES that can be parsed as valid molecules by RDKit
Canonical Exact Match	Measures how many generated molecule exactly matches the target product

We found the best configuration using RayTune based on a combined validation score that prioritized exact match and valid SMILES generation. After tuning, the selected model configuration was used for final training and evaluation. The results show that hyperparameter tuning improves all parameter of the results.

Table 3: Best Hyperparameter Configuration for GNN+Transformer

Hyperparameter	Selected Value
Hidden Dimension	256
Number of GNN Layers	2
Number of Decoder Layers	4
Dropout Rate	0.1
Learning Rate	0.000284254
Batch Size	8

4.5 Transformer

For our final sequence-to-sequence model, we implemented an encoder-decoder transformer with shared token embeddings between source and target sequences. As opposed to character-level or atom-level tokens, this transformer uses SmilesPE tokens [8], which aim to learn the structures and local grammar of the molecules they represent. SmilesPE was originally trained on a ChEMBL: a large-scale bioactivity database for drug discovery [4]. SmilesPE uses unsupervised learning to group commonly represented segments of varying length. It prioritizes these segments using a rank learned from the frequency of segments during training. To tokenize a string, it tokenizes segments in order of rank, generally starting with larger chunks and filling in the gaps with smaller tokens like single atoms.

Reactant and product SMILES strings are tokenized using the pretrained SmilesPE tokenizer, embedded into a D-dimensional space (chosen through hyperparameter tuning), and augmented with sinusoidal positional encodings before being processed by the transformer. This method allows us to take the grammar learned from millions of rows of drug-based chemical data and apply it to our specific use case through the embeddings learned during training. The model uses 4-layer encoder and decoder blocks with multi head self-attention, feed-forward networks, residual connections, and dropout regularization. The model is trained with a token-level cross entropy loss using teacher forcing, where ground truth tokens are provided as the precursor to each subsequent token prediction. An AdamW optimizer with gradient clipping was chosen to improve stability and help mitigate exploding gradients. During training, the learning rate is reduced as the exact-match accuracy on the validation dataset plateaus. This exact-match accuracy is collected every 3 epochs to reduce slowdowns during training caused by converting tokens back into SMILES strings, canonicalizing the strings, and comparing to the canonicalized target strings for each reaction. Since this complex string manipulation is not well suited for GPUs, reducing the frequency of evaluations improves training speed by significant margins.

Hyperparamter tuning is performed using RayTune with random search across predetermined distributions to select the learning rate, batch size, dropout rate, embedding dimensions, and feed-forward network size to optimize validation loss. Subsets of 50,000 training rows and 5,000 validation rows are used to train 10 models with different hyperparameter configurations over 5 epochs to determine the final parameters for the model. Then the model is trained on the entire training dataset for 30 epochs, keeping track of the model with the best exact-match validation accuracy. Table 4 shows the search space for RayTune's random search. All later transformers run on augmented datasets are also tuned using the exact hyperparameter options using a single seed.

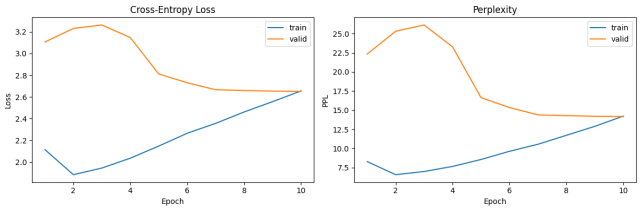

Figure 1: Training and Validation loss and Perplexity

Table 4: Transformer Hyperparameter Search Space

Hyperparameter	Search Space
Learning rate	Log-uniform: $[1 \times 10^{-5}, 5 \times 10^{-4}]$
Batch size	$\{32, 64, 128\}$
Dropout rate	Uniform: $[0.0, 0.3]$
Embedding dimension	$\{128, 256, 384\}$
Feed-forward dimension	$\{512, 1024, 2048\}$

Ultimately, the following hyperparameters were chosen for the transformer:

- Batch size: 64
- Learning rate: 2.34×10^{-4}
- Dropout: 0.0156
- Embedding dimension: 384
- Feed Forward dimension: 2048

5 Results

5.1 Baseline

After tuning hyperparameters with Optuna [1], the baseline recurrent neural network produced a 0% exact match accuracy, 18.1% token accuracy, and 0% valid SMILES prediction rate.

After 10 epochs, the final training, validation, and testing loss were all 2.65. Notably, the training loss initially decreased, then began increasing after epoch 3; whereas, the validation loss initially increased, then began decreasing after epoch 3. The training loss behavior can be attributed to the decrease in teacher forcing. As the model stops receiving help from the teacher, it begins to make more mistakes. Since SMILES contains many tokens and all the information is funneled into a single hidden state, the vanilla RNN's hidden state eventually collapsed. As a result, it decided to continuously predict the most common tokens to minimize the loss. This explains why the accuracies and valid SMILES prediction rate are so low, despite having reasonable loss.

5.2 Recursive Neural Network

The Recursive NN substantially improved token-level prediction quality compared with the baseline recurrent architecture. During training, both training and validation loss decreased steadily, indicating stable optimization behavior without severe overfitting.

After training, the final validation loss reached 0.2948. Training token accuracy achieved 88.43%, while validation token accuracy reached 88.27%. Figure 2 shows that both training and validation loss decreased steadily across epochs, suggesting stable optimization. Figure 3 shows that token accuracy improved consistently throughout training.

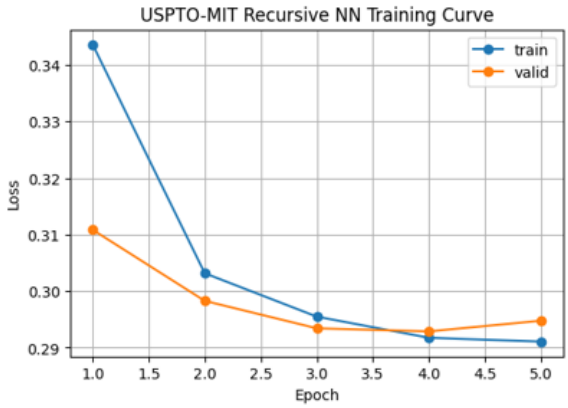

Figure 2: Training and validation loss across epochs for the Recursive Neural Network.

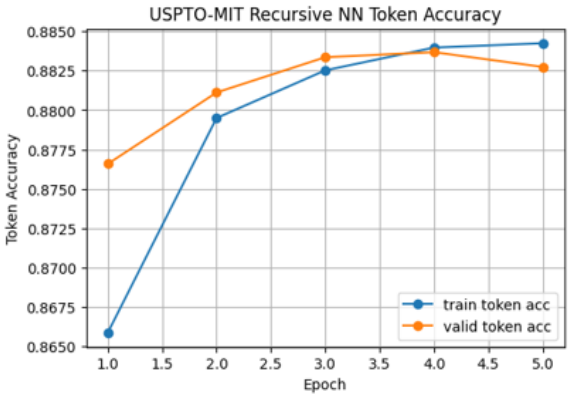

Figure 3: Training and validation token accuracy across epochs for the Recursive Neural Network.

Although token-level performance improved substantially (Figure 3), exact-match performance remained at 0%. This result highlights a key challenge in chemical reaction prediction: predicting individual SMILES tokens correctly does not necessarily lead to generating an entirely correct product molecule.

Because SMILES strings are highly sensitive to small decoding errors, a single incorrect token may invalidate the entire product prediction even when most of the sequence is correct. The Recursive NN therefore learned local syntactic and structural patterns but struggled to preserve full molecular consistency across long output sequences.

These results suggest that character-level sequence generation alone is insufficient for high-quality reaction prediction and motivate stronger architectures such as graph-based encoders, molecular tokenization methods, and transformer-based decoders.

5.3 Graph Neural Network

The training results for the GNN model showed that the model converged relatively quickly, with no signs of overfitting. The loss curve can be seen in Figure 4, with both training and validation loss declining quickly for the first few epochs, then leveling off after around epoch 6. The fact that the validation loss remains approximately equal to the training loss through all 10 epochs indicates that the model is not overfitting the data.

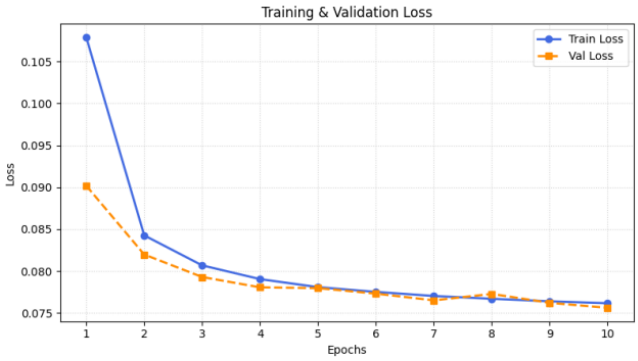

Figure 4: Training and validation loss for the simple GNN model.

Figure 5 shows the validation accuracy, precision, and recall, which were around 98%, 65%, and 95%, respectively. Precision and recall are vital metrics for this analysis, because although the accuracy value appears to be very high, it does not tell the full story. Most of the predictions that the model makes are easy predictions, as most pairs of atoms are not bonded, which drives up the accuracy score. By analyzing the precision and recall values, we can see that the model is over-predicting: almost all of the bonds that are supposed to be present in the target product are being successfully predicted, but the model is also predicting many additional bonds that are not supposed to be present in the true product.

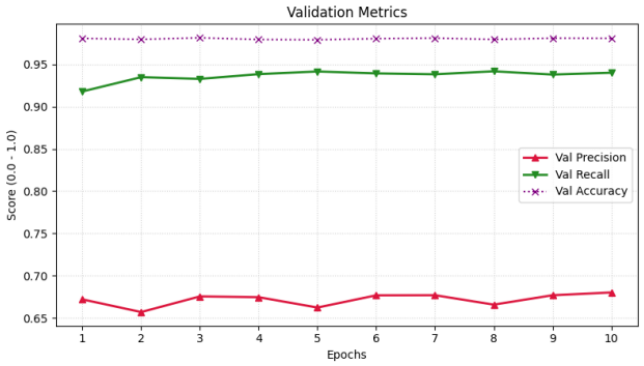

Figure 5: Validation accuracy, precision, and recall for the simple GNN model.

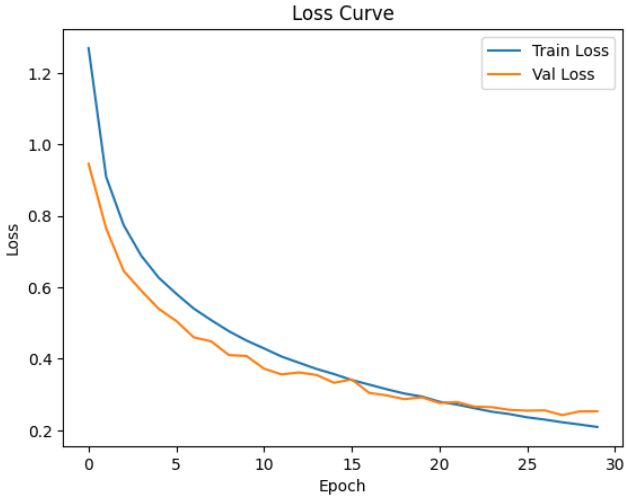

Figure 6: Training and Validation loss for GNN + Transformer

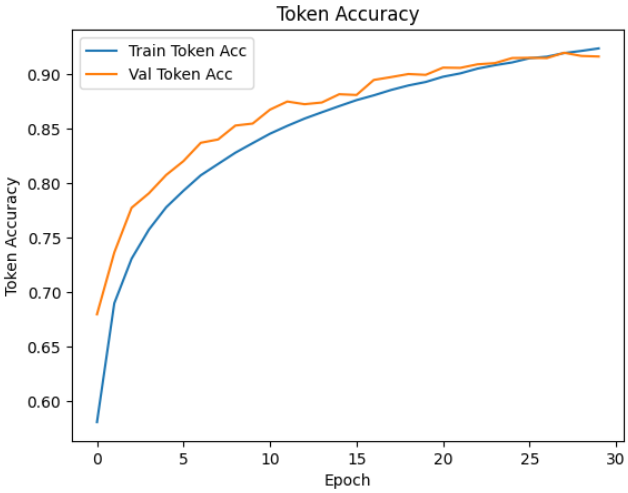

Figure 7: Training and Validation Token Accuracy for GNN + Transformer

We conducted further model evaluation by running the model on the test set of data. For this evaluation, we used the predictions from the GNN to actually generate the predicted graphs, then compared these predicted graphs to the true graphs of the products, calculating a Jaccard similarity score [5] to quantify the similarity between the graphs. The average graph similarity score was 0.75, and the exact match accuracy was 3.91%. We also found that only 41.1% of the predicted graphs were chemically valid, which shows that the model did not adhere strongly to the laws of chemistry when making predictions, largely due to overloading atoms with more bonds than are physically possible, due to the over-prediction issue described previously. While it is possible to adjust the loss function to bring the validation precision and recall into balance, thereby addressing the over-prediction issue, we found that doing so resulted in a severe degradation of the performance of the model, driving the average graph similarity below 0.5, and the exact match accuracy below 0.3%; therefore, such a modification is evidently not worthwhile.

These results show that the GNN model outperformed the baseline RNN and the recursive NN models in exact match accuracy, capturing the overall structure of chemical reactions, but failed to truly learn the intricate details that would be necessary to make accurate enough predictions for practical use. An exact match accuracy as low as 3.91% demonstrates the need for more thorough and advanced approaches to handle the task of chemical reaction prediction.

5.4 Graph Neural Network + Transformer

The results show that the model successfully learns the SMILES syntax. The training loss decreases to 0.2092, and the training token accuracy reaches 0.9239. The validation loss is 0.2532, and the validation token accuracy reaches 0.9164. This indicates that the model is able to learn meaningful SMILES patterns rather than producing completely random sequences. (Figure 6) Also, it can predict the next SMILES token, which shows that model learns the SMILE syntax. (Figure 7) However, it does not mean that it is all valid SMILES because it predict the next SMILE token and it can lead not closing the ring number.

The valid SMILES rate reaches approximately 0.65, meaning that more than half of the generated products can be parsed as chemically valid molecules by RDKit. This means that the Transformer decoder learned important SMILES syntax rules, such as atom tokens, branches, ring structures. However, the canonical exact match reaches 6%. This means that even when the generated SMILES products are chemically valid, they do not exactly match the target product molecules after canonicalization. This means that the model is able to learn the pattern of SMILES and the molecules graph, but it is hard to learn the chemical reaction between reactants. (Figure 8)

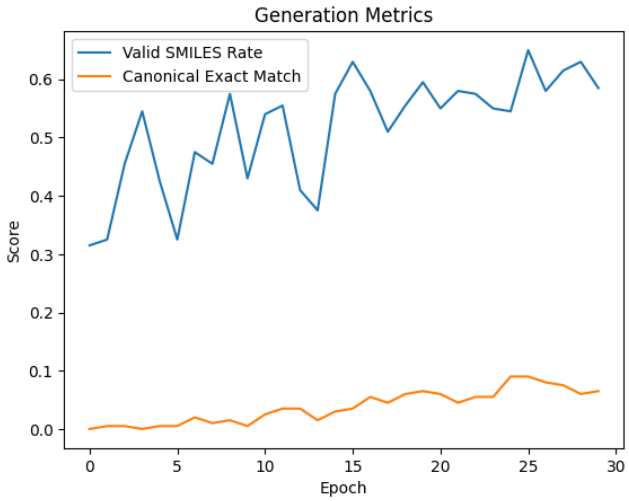

Figure 8: Valid SMILES and EXACT MATCH for GNN + Transformer

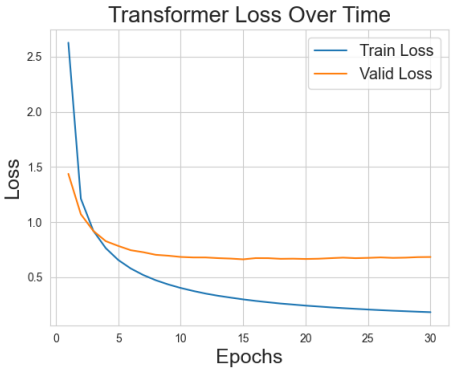

Figure 9: Loss metrics for transformer model

As seen in Figure 9, the validation loss is beginning to plateau, but it is not increasing yet. With loss being token loss as opposed to exact-match loss, it can still be improving at exact-match prediction while the token loss stays relatively stable. This can be seen in Figure 10, where the three accuracy metrics are still noticeably increasing at the 30 epoch threshold.

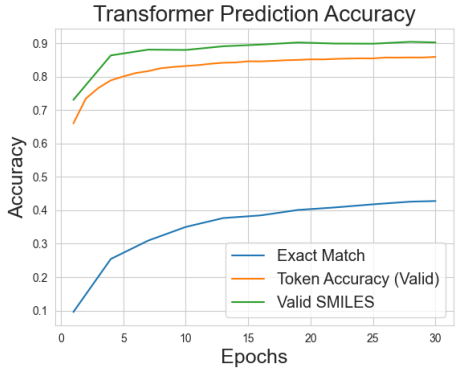

Figure 10: Accuracy metrics for transformer model

Overall, the model is somewhat successful. It learns how to generate the molecule using SMILES syntax and predict the next SMILES token. However, it still struggles with exact reaction product prediction. This shows that the model needs to improve learning the chemical reaction step by step.

For future improvement, The model will train the chemical reaction pathway dataset step by step, not just reactants and products. In the current approach, the model receives the reactants and generates the final product SMILES directly. However, chemical reaction process works in various stages, such as bond breaking and bond formation. If the model can train to understand the reaction mechanism process, then it will learn how reactants gradually change into products, rather than a direct input and output mapping.

5.5 Transformer

While it would be ideal to learn the mechanisms of chemical reaction, the transformer shows that it is not necessarily crucial in order to successfully predict reactions. The transformer managed to perform significantly better than the other explored models at exact match prediction. While most other models had an exact-match accuracy of around 0%, the SmilesPE transformer managed to achieve an exact-match accuracy of 43.8% on the test dataset after 30 epochs of training. This is not just a result of training for more time, because even after 4 epochs, the transformer managed to achieve an exact match accuracy of 25.4% on the validation set. While 43.8% is still not ideal for predictions, it is a large improvement on the baseline and other models. An improvement this substantial on the reaction task is not trivial, especially with how unforgiving exact-match accuracy can be for large chemical reactions.

Despite the relatively constant validation loss, the training loss and the various accuracy metrics are improving. These accuracy metrics confirm that the model is not overfitting when its training loss is decreasing. Instead, the validation token loss does not show the entire picture of the model's improvement. Notably, the valid SMILES rate is much higher than the exact match accuracy. Thus, even when the model does not get the correct answer, it still seems to have learned how molecules are structured, rather than blindly guessing tokens. Now that we have seen the transformer outperform the other models, we will see how it generalizes to an unseen dataset and we will try to improve the robustness of the model through data augmentation.

6 Transformer Generalization Experiments

While the exact match accuracy of the transformer is unrivaled by our other models, the token accuracy does not outperform the recursive neural network on paper. However, these two metrics are

not strictly comparable due to their different definitions of token. Because the recursive neural network uses character-level tokens, the number of total choices to choose from is far lower. This makes single token prediction far easier than the transformer's SmilesPE token prediction. So, although the recursive neural network has a higher value for token accuracy, it does not have a better predictive power; it is a side effect of the different definitions of tokens.

Further examining the SmilesPE tokens, it could be possible that the SmilesPE tokens do not translate well to the USPTO-MIT dataset, since they were trained on the ChEMBL dataset. To test these results, we can take a quantitative and qualitative look at the two datasets and see if the tokens transfer well. Note that the tokens themselves are the only parts that need to transfer, as the embeddings are learned from the USPTO-MIT dataset.

Comparing the two datasets in terms of their makeup, the ChEMBL dataset is primarily for drug discovery, so it has a high quantity of pharmaceutical chemicals [4]. From the description of USPTO-MIT, we see that it does also contain many pharmaceuticals as well as other chemicals for polymers and pesticides. Since there is a large overlap here, holistically it seems to be a fine fit.

Moving on to a quantitative view, we can count how many of the unique SmilesPE tokens are used in our training data. In total, there are 2814 unique tokens in the downloaded version of SmilesPE, and in our training set, 2805 of those tokens are represented. Thus, only 9 out of over 2800 tokens are not transferable, further suggesting that SmilesPE is a fine fit for a tokenizer.

Finally, looking at the distribution of the SmilesPE tokens in the training set, we can see in Figure 11 that they follow a natural distribution.

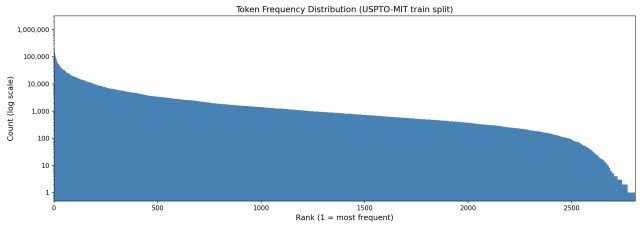

Figure 11: SmilesPE Token Distribution in Training Data

Note the y-axis is log-scaled, so a small portion of unique tokens account for a large proportion of the total token frequencies, and then it smoothly travels out from there, with a large portion of tokens having roughly similar frequencies to each other throughout the training data. This is similar to what we would expect from analyzing English words in spoken or written language. Words such as "the", "and", pronouns, and other prepositions would appear very frequently, but there would not be a drastic drop where all other words have essentially no representation. If there were a dramatic drop like that in our figure, we would have to reconsider using SmilesPE for this dataset, however due to the natural distribution, it seems that SmilesPE is a good fit.

To increase the robustness of our model to unseen data, one commonly used technique is SMILES randomization [2]. A single molecule can be represented by multiple valid SMILES strings. As such, we can augment the training data to take a few random samples from those valid string possibilities for each source reaction. We maintain canonicalized targets so that we don't accidentally bias the model towards outputting a certain type of randomized target when it sees a randomized source. Also, this reinforces the idea that one molecules can be represented multiple different ways; the inputs change but the outputs remain the same.

For the experiment of random SMILES data augmentation, we keep the transformer architecture the same, and rerun the hyperparameter tuning and subsequent training on two new datasets: the training set with 1 additional randomized SMILES reaction per standard reaction, and one with 4 additional randomized rows. This way we can see how a smaller or larger number of augmentations affect the results. For each experiment, first the data augmentation is performed, then the data is tokenized, embeddings are created, and the model is trained. Each transformer is still run for 30 epochs, keeping track of the model that performs the best on the validation exact match. For all 3 models (0 augmentations, 1 augmentation, and 4 augmentations), the best epoch was the final epoch. This once again indicates that the models would benefit from continued training. However, due to time and compute constraints, 30 epochs was the max that could be done.

After training the models, they are all evaluated on the USPTO-MIT test dataset. Table 5 shows the results of this experiment.

Table 5: USPTO-MIT Performance of Transformers Trained with Varying SMILES Augmentation

Model	Exact Match	Token Acc	Valid SMILES
0 Augmentations	43.80%	63.8%	90.4%
1 Augmentation	43.83%	64.8%	93.4%
4 Augmentations	36.1%	58.8%	95.7%

From Table 5, we see some relatively surprising results with the exact match accuracy. The exact match percentage of the model trained on 1 augmentation and the original transformer are nearly identical. Furthermore, the transformer trained with 4 random augmentations has a significant drop in exact match accuracy. However, looking at the valid SMILES rates, the results become more clear: randomized SMILES augmentations seem to improve the models' ability to understand the grammar and syntax of molecules, but it does not necessarily translate to improved reaction prediction accuracy. From 0 to 1 to 4 augmentations, we see a clear increase in valid SMILES prediction rates. While the token accuracy is 1% higher in the 1 augmentation model compared to the original transformer, the lack of improvement in exact match accuracy shows that the models trained on augmented data are not improving at chemical reaction prediction in this dataset. Instead, they become more adept at understanding how SMILES strings work.

Finally, we test the 3 transformer models on a completely new dataset to see how well they generalize. There is no training done on the new dataset; the tokens, embeddings, and weights are exactly the same. The only change is the reactions that the models are tested on.

Table 6: Performance of USPTO-Trained Transformers Evaluated on the ORDerly Dataset

Model	Exact Match	Token Acc	Valid SMILES
0 Augmentations	42.7%	59.1%	89.2%
1 Augmentation	44.3%	60.2%	93.1%
4 Augmentations	37.1%	58.6%	94.9%

Looking at the results from Table 6, we can see that the models generalize surprisingly well. Compared to the results from the USPTO-MIT tests, these are overall slightly lower, but usually by only around 1%. The 4 augmentations model did slightly improve on the exact match accuracy, indicating that the added robustness did seem to help it generalize to unseen data. However, it is still overall lower than the other two, so it does seem like the high quantity of augmented data did overall hurt the model. With the 1 augmentation and 0 augmentation models being quite close to each other in the various accuracies, it seems that the augmentation helps slightly to generalize to new data, but more to improve valid SMILES rates than exact match prediction. Overall, the models are seemingly robust to new data, at least in the realm of organic chemistry, but they still have a lot of room for improvement in terms of exact match accuracy.

7 Conclusion and Discussion

Overall, most of our models did not obtain the exact match accuracy that we had hoped for. The baseline RNN showed that this reaction problem is far more difficult than a rudimentary model with limited memory or attention can handle. The recursive neural network improved on the token accuracy substantially, but it was still not able to learn the underlying chemistry in order to successfully predict exact match reactions. We approached the problem with some graph models to see if they could improve upon the exact match accuracy, and while they did improve to a certain degree, the accuracy was still quite low. The basic GNN was not easily comparable to the other models since it predicts edges rather than final sequences, however it did manage to achieve a basic understanding of which bonds would be in the final result. The Graph+Transformer model was an attempt at creating a graph model with the ability to produce sequence outputs that would hopefully be able to learn from the underlying chemical bonds and features. While it did show some promise, it ultimately did not perform as well as expected. There is a chance that it had difficulty converting between understandings of latent graph representations and decoding that into a final product, or it could be for other reasons such as not being a complex enough graphical picture.

As we mentioned in the graph Encoder-Decoder model, training with chemical reaction pathway datasets and predicting each step of chemical reactions may improve the exact match accuracy. This could teach the model how the reactants transform into products step by step and inform the chemical reaction processes. The graph we use is a 2-dimensional graph. If we use a 3-dimensional graph, the model may be able to consider the exact location of atoms and bonds as they appear in 3D space. There are many hidden factors for chemical reaction, so if the model learns the process and pattern of chemical reactions, it is likely that its predictive power will improve.

The SmilesPE transformer was a major improvement with exact match accuracy, achieving up to 43.8% accuracy, however it could still be more accurate. The training was cut short at 30 epochs due to time and compute constraints, and there are more methods for improving accuracy that could be explored in further works. One such example is to use a beam-search with the transformer to get a top-K prediction rather than a top-1 prediction. Other works [11] used transformers to achieve much higher top-1 reaction predictions (90%), although they were trained on more and larger datasets for longer periods of time. There is some evidence to believe that with more time training, and especially more novel training data, that our transformer model could achieve higher accuracy levels. Another improvement could be to change the loss function to be exact-match loss rather than token-loss, however, this would require much more time for training. Overall, there are several future directions to improve our transformer models, but they generally require more time and compute. With the time and computational power that we did have, we were able to achieve far better results from our transformer models than our other models, and we showed that they were quite generalizable to unseen data.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. arXiv:1907.10902 [cs.LG] https://arxiv.org/abs/1907.10902

[2] Josep Arús-Pous, Simon V. Johansson, Oleksandr Prykhodko, Esben Jannik Bjerrum, Christian Tyrchan, Jean-Louis Reymond, Hongming Chen, and Ola Engkvist. 2019. Randomized SMILES strings improve the quality of molecular generative models. *Journal of Cheminformatics* 11, 1 (2019), 71. doi:10.1186/s13321-019-0393-0

[3] Matthias Fey and Jan Eric Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. *CoRR* abs/1903.02428 (2019). arXiv:1903.02428 http://arxiv.org/abs/1903.02428

[4] Anna Gaulton, Louisa J. Bellis, A. Patricia Bento, Jon Chambers, Mark Davies, Anne Hersey, Yvonne Light, Shaun McGlinchey, David Michalovich, Bissan Al-Lazikani, and John P. Overington. 2012. ChEMBL: a large-scale bioactivity database for drug discovery. *Nucleic Acids Research* 40, D1 (2012), D1100–D1107. doi:10.1093/nar/gkr777

[5] Paul Jaccard. 1901. Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles* 37 (1901), 547–579.

[6] Wengong Jin, Connor Coley, Regina Barzilay, and Tommi Jaakkola. 2017. USPTO-MIT Reaction Dataset. https://github.com/wengong-jin/nips17-rexgen/raw/master/USPTO/data.zip.

[7] G Landrum. [n. d.]. RDKit. https://www.rdkit.org

[8] Xinhao Li and Denis Fourches. 2021. SMILES Pair Encoding: A Data-Driven Substructure Tokenization Algorithm for Deep Learning. *Journal of Chemical Information and Modeling* 61, 4 (2021), 1560–1569. PMID: 33715361. arXiv:https://doi.org/10.1021/acs.jcim.0c01127 doi:10.1021/acs.jcim.0c01127

[9] Richard Liaw, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E. Gonzalez, and Ion Stoica. 2018. Tune: A Research Platform for Distributed Model Selection and Training. arXiv:1807.05118 [cs.LG] https://arxiv.org/abs/1807.05118

[10] Sanggil Park, Herim Han, Hyungjun Kim, and Sunghwan Choi. 2022. Machine Learning Applications for Chemical Reactions. *Chemistry – An Asian Journal* 17, 14 (2022), e202200203. arXiv:https://aces.onlinelibrary.wiley.com/doi/pdf/10.1002/asia.202200203 doi:10.1002/asia.202200203

[11] Philippe Schwaller, Teodoro Laino, Théophile Gaudin, Peter Bolgar, Christopher A. Hunter, Costas Bekas, and Alpha A. Lee. 2019. Molecular Transformer: A Model for Uncertainty-Calibrated Chemical Reaction Prediction. *ACS Central Science* 5, 9 (2019), 1572–1583. PMID: 31572784. arXiv:https://doi.org/10.1021/acscentsci.9b00576 doi:10.1021/acscentsci.9b00576

[12] David Weininger. 1988. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences* 28, 1 (1988), 31–36. arXiv:https://doi.org/10.1021/ci00057a005 doi:10.1021/ci00057a005

[13] Daniel S. Wigh, Joe Arrowsmith, Alexander Pomberger, Kobi C. Felton, and Alexei A. Lapkin. 2024. ORDerly: Data Sets and Benchmarks for Chemical

Reaction Data. *Journal of Chemical Information and Modeling* 64, 9 (2024), 3790–3798. PMID: 38648077. arXiv:https://doi.org/10.1021/acs.jcim.4c00292 doi:10.1021/acs.jcim.4c00292

8 Appendix

The code for our project can be found at https://github.com/ravishende/neural_networks_final_project

Contributions

Andrew Zheng. Implemented the baseline vanilla RNN model

Jae Choi. Implemented the GNN + Transformer model (encoder - decoder)

Paul Schroller. Implemented, refined, trained, and evaluated the simple GNN model, and wrote the sections about this model. Helped with researching datasets. Created helper functions to load and process the ORDerly benchmark dataset.

Qihua Xue. Implemented and trained the Recursive Neural Network sequence-to-sequence pipeline for chemical reaction prediction using the USPTO-MIT dataset. Developed preprocessing and training workflows, conducted hyperparameter tuning with Ray Tune, analyzed training and validation performance, and generated evaluation metrics including token accuracy and exact-match accuracy. Helped with researching dataset. Wrote sections on Recursive Neural Network.

Ravi Shende. Implemented and trained the SmilesPE transformer with hyperparameter tuning and custom embeddings. Researched different datasets to use, various ways to improve accuracy. Created helper functions and notebooks to efficiently run the experiments of multiple models with various augmentation levels. Implemented SMILES augmentation, ability to pull model from USPTO and test on ORDerly. Ran analysis of SmilesPE and researched various methods of tokenization. Wrote the abstract for the paper and added deeper explanations and consequences of token accuracy and exact match accuracy in introduction. Wrote sections on Transformer architecture, Transformer results, and generalization experiments. Added all citations, wrote section on Datasets. Helped write conclusion and discussion section.